

RELATÓRIO DE FLUXO - ANALISADOR FINANCEIRO

Gerado em: 17/08/2026 15:27

Perfeito. O entendimento está confirmado. Com base no `App\_Completo.txt` (que contém todos os seus módulos) e no JSON de exemplo, mapeei integralmente o fluxo de dados, as dependências e os scripts órfãos.

Abaixo está o **relatório completo em tabelas**, estruturado em quatro blocos:

- Coleta** – origens brutas e periodicidade.
- Processamento** – transformações e encadeamento.
- Consumidores Finais (UI)** – quais páginas utilizam quais JSONs.
- Scripts Inativos / Órfãos** – com sugestão de ação (deletar ou mover para reserva).

1. TABELA DE COLETA (DADOS BRUTOS)

Script	O que coleta	Fonte (De onde)	Frequência	Quem consome (próximo elo)
Coletor.py	PTAX oficial (USD_BRL) • Ajuste B3 (WIN_AJUSTE / WDO_AJUSTE) • 21 ativos globais (VIX, S&P, NASDAQ, DXY, ADRs, Minério, Petróleo, etc.) • Último tick (last) via MT5 (WIN/WDO) – na janela de ajuste BACEN SGS (API), TradingView Scanner API, MetaTrader 5 (v2.2 preferencial / v1 fallback) A cada 5 min (disparado pelo Agendador) Validador.py e Limpar_Imagens_TradingView.py (paralelo)			
Coleta_Noticias_Calendario.py	Calendário econômico Brasil e EUA (eventos 2★ e 3★) TradingView Economic Calendar API 1 vez ao dia (cache reutilizado se já coletado hoje) Analise_Noticias.py			
Coletor_MT5_v2_2.py	(preferencial) Coletor_MT5.py (fallback) Last / Bid / Ask / Volume dos contratos WIN, WDO, DI1 (seleção dinâmica do contrato mais líquido) MetaTrader 5 (terminal) Apenas quando chamado pelo Coletor.py (dentro da janela de ajuste 19:00–08:50) Coletor.py (integração via capturar_last_do_mt5())			

2. TABELA DE PROCESSAMENTO (TRANSFORMAÇÕES E AGREGAÇÕES)

Script	Informação gerada	Insumos (dependências)	Frequência	Quem consome o resultado
Limpar_Imagens_TradingView.py	Gerencia imagens de gráficos (move as 2 mais recentes para Coletas/WIN_1min.png e WIN_5min.png) Imagens baixadas na pasta Downloads do usuário Executado no pipeline (etapa 0) Páginas de análise visual (UI) – 7_ IA_Imagem.py, 7_ IA_Gemini.py, 8_ Gerador_Profit_Pro.py			
Validador.py	Dados_Validados.json (sanitiza e padroniza os 24 ativos) Coleta_rom-0.json A cada 5 min (pipeline) Calculadora.py, CalculadoraEstimativaAbertura.py			
Analise_Noticias.py	Noticias_Impacto_Dia.json (classificação de risco, alertas 3★, múltiplas 2★) Noticias_Calendario.json 1x ao dia (ou quando chamado) Engine_Vies.py, UI (pages/13_ Noticias.py)			
MapearTendencia15Min.py	Analise_Tendencias.json (comportamento das últimas 3 coletas: rom-10 → rom-5 → rom-0) Coleta_rom-10.json, Coleta_rom-5.json, Coleta_rom-0.json A cada 5 min (pipeline) Engine_Vies.py, UI (pages/6_ Analise_Tendencia.py, pages/1_ Setup_Abertura.py)			
Calculadora.py	Metricas_Calculadas.json (spreads, curva DI, VIX, indicador mercado externo, indicador ADRs) Dados_Validados.json A cada 5 min (pipeline) Engine_Vies.py, UI (pages/4_ Calculadora.py, pages/5_ Core_Engine.py)			
CalculadoraEstimativaAbertura.py	EstimativaAbertura.json (abertura teórica WIN/WDO, pivôs, resumo macro) Dados_Validados.json A cada 5 min (pipeline) Engine_Vies.py, Gerar_Resultado_Operacional_Abertura.py, UI (pages/4_ Calculadora.py)			
Motor_SMC_Regras.py	(via Rodar_SMC_Regras.py) AnaliseGraficaSMC_Regras.json (OB, FVG, BOS/CHoCH, liquidez, níveis) MetaTrader 5 (candles 5min) A cada 5 min (pipeline) Engine_Vies.py (via UI/prompt), UI (pages/9_ SMC_Regras.py, pages/1_ Setup_Abertura.py)			
Engine_Vies.py	Decisao_Core.json (viés final e score para WIN e WDO, com fatores relevantes) EstimativaAbertura.json, Metricas_Calculadas.json, Noticias_Impacto_Dia.json, DadosAtivosUnificados.json, Analise_Tendencias.json, AnaliseGraficaSMC_Regras.json A cada 5 min (pipeline) UI (pages/5_ Core_Engine.py, pages/1_ Setup_Abertura.py), Gerar_Resultado_Operacional_Abertura.py			
Gerar_Resultado_Operacional_Abertura.py	Resultado_Calculadora_Operacional_Abertura.json (consolida todos os indicadores + classificação operacional) Metricas_Calculadas.json, EstimativaAbertura.json, Decisao_Core.json, DadosAtivosUnificados.json, Analise_Tendencias.json, Noticias_Calendario_0900.json A cada 5 min (pipeline) UI (pages/1_ Setup_Abertura.py, pages/4_ Calculadora.py)			

`Gerar\_Relatorio.py`	`Relatorio\_Executivo.md` (relatório macro em Markdown)	`Metricas\_Calculadas.json`	A cada 5 min (pipeline)	UI (exibição textual)
`Gerar\_Relatorio\_Mensagem.py`	Relatório de abertura (texto)	`EstimativaAbertura.json`	A cada 5 min (pipeline)	UI (utilizado em algumas páginas como resumo)
`v2\_gravar\_sessao\_win.py`	Histórico de sessão em `Coletas/Historico\_Aberturas/YYYY-MM-DD.json` (usando a nova arquitetura V2)	Todos os dados via `build\_win\_session()` + `opening\_scenario\_engine`	A cada 5 min (pipeline)	UI V2 (`pages/20\_█\_Previsao\_Abertura\_WINFUT.py`, `pages/21\_█\_Historico\_Macro.py`)
`Gerar\_Mapa\_Projeto.py`, `Gerar\_Mapa\_Fluxo.py`, `Gerar\_ArquivosApp.py`	Mapas de documentação (`Mapa\_Projeto.json`, `Mapa\_Fluxo.json`, `ArquivosApp.py`)	Escaneamento do sistema de arquivos (sem insumos de dados)		
****Manual / sob demanda**** (não está no pipeline operacional)	UI (`pages/15\_██\_Mapa\_da\_Aplicacao.py`)			

3. TABELA DE CONSUMIDORES FINAIS (INTERFACE / DASHBOARDS)

Estas páginas ****não produzem dados para outras etapas****; apenas leem os JSONs gerados para exibição.

Página Streamlit	JSONs / Dados que consome
`pages/1\_█\_Setup\_Abertura.py`	`Noticias\_Calendario\_0900.json`, `Metricas\_Calculadas.json`, `EstimativaAbertura.json`, `Decisao\_Core.json`, `DadosAtivosUnificados.json`, `Analise\_Tendencias.json`, `Resultado\_Calculadora\_Operacional\_Abertura.json`, `AnaliseGraficaSMC\_Regras.json`
`pages/4\_█\_Calculadora.py`	`EstimativaAbertura.json`, `Resultado\_Calculadora\_Operacional\_Abertura.json`, `Metricas\_Calculadas.json`, `DadosAtivosUnificados.json`, `Analise\_Tendencias.json`
`pages/5\_██\_Core\_Engine.py`	`Decisao\_Core.json`, `Dados\_Validados.json`, `Noticias\_Impacto\_Dia.json`, `Metricas\_Calculadas.json`, `Analise\_Tendencias.json`, `EstimativaAbertura.json`
`pages/6\_█\_Analise\_Tendencia.py`	`Analise\_Tendencias.json`
`pages/9\_█\_SMC\_Regras.py`	`AnaliseGraficaSMC\_Regras.json`, `AnaliseGraficaSMC.json` (visão IA, se existir)
`pages/13\_█\_Noticias.py`	`Noticias\_Impacto\_Dia.json`, `Noticias\_Calendario.json`
`pages/14\_█\_Ativos\_Monitorados.py`	`DadosAtivosUnificados.json`
`pages/15\_██\_Mapa\_da\_Aplicacao.py`	`Mapa\_Projeto.json`, `Mapa\_Fluxo.json`, `Pipeline\_Log.json`
`pages/16\_█\_Dados\_MT5.py`	`Dados\_MT5.json` (via `Coletor\_MT5`)
`pages/18\_█\_Previsao\_Inteligente\_Abertura.py`	Dados do novo motor (via `NOVO\_MOTOR\_PREVISAO\_ABERTURA/core/motor\_previsao.py`)
`pages/19\_█\_Previsao\_Inteligente\_Abertura\_Comparador.py`	Mesmo do 18 + comparação com legado (`Decisao\_Core.json`)
`pages/20\_█\_Previsao\_Abertura\_WINFUT.py`	V2 (`build\_win\_session()`, `opening\_scenario\_engine`)
`pages/21\_█\_Historico\_Macro.py`	`Coleta\_rom-\*.json` (leitura direta dos ROMs)
`pages/7\_█\_IA\_Gemini.py`	Imagens (`WIN\_1min.png`, `WIN\_5min.png`) – gera `AnaliseGraficaSMC.json` (visão IA)
`pages/7\_█\_IA\_Imagem.py`	Imagens + dados do pipeline (via `KeyManager` e leitura dos JSONs)
`pages/8\_█\_Gerador\_Profit\_Pro.py`	`AnaliseGraficaSMC\_Regras.json`, `AnaliseGraficaSMC.json` ou texto colado

4. TABELA DE SCRIPTS INATIVOS / ÓRFÃOS (NÃO UTILIZADOS NO PIPELINE OPERACIONAL)

Script	Motivo / Status	Ação sugerida
`Coletor copy.py`	Cópia exata do `Coletor.py`. Não é chamado por ninguém.	███ ****Deletar**** (redundante)
`Monitor\_WIN\_Leilao.py`	Monitor avulso de leilão (não integrado ao pipeline).	█ ****Mover para pasta `\_reservados`****
`Teste\_Book\_WIN.py`, `Teste\_Historico\_Ticks\_WIN.py`, `Teste\_Leilao\_MT5.py`, `Teste\_MT5.py`, `Teste\_Vencimento\_MT5.py`	Scripts de teste/debug do MT5. Não chamados por nenhum módulo.	█ ****Mover para `\_reservados/testes`****
`test\_import.py`, `teste\_motor.py`	Testes de importação do `KeyManager` e do novo motor.	█ ****Mover para `\_reservados/testes`****
`diagnostico\_env.py`	Diagnóstico de variáveis de ambiente (.env).	█ ****Mover para `\_reservados/utilitarios`****
`Gerar\_ArquivosApp.py`, `Gerar\_Mapa\_Inventario\_Tecnico.py`	Geradores de inventário (substituídos por `Gerar\_Mapa\_Projeto.py` e `Gerar\_Mapa\_Fluxo.py`).	███ ****Deletar**** ou █ ****Mover para `\_reservados/legado`****
`pages/3\_█\_Setup\_Abertura\_10h.py`	Página separada para setup das 10h (não linkada no menu principal do `app\_home.py`). Aparentemente obsoleta (unificada na `pages/1\_...`).	█ ****Mover para `\_reservados/pages\_antigas`****
`pages/8\_█\_Gerador\_Profit\_Pro\_Original.py`	Cópia antiga do gerador Profit Pro (substituída pela versão atual `8\_█\_Gerador\_Profit\_Pro.py`).	███ ****Deletar****

| `pages/20\_■\_Previsao\_Abertura\_WINFUTa.py` e `...s.py` | Versões intermediárias da página V2 (não linkadas). | ■ ****Mover**
para `\_reservados/pages\_antigas/`** |
| `pages/21\_■\_Historico\_Macro copy.py` e `copy 2.py` | Cópias antigas da página de histórico macro. | ■■ ****Deletar**** ou ■
****Mover para `\_reservados/pages\_antigas/`**** |

Resumo executivo:

- O pipeline tem ****8 etapas de coleta/processamento ativas**** que se encadeiam corretamente.
- A maioria dos scripts órfãos são ****testes, duplicatas ou utilitários de documentação**** que não afetam a operação.
- Recomendo criar a pasta `\_reservados/` na raiz e mover todos os listados como "**■ Mover**" para manter o projeto limpo, sem perdê-los.